Least Squares

Minimize || A x - b ||

if there are solutions then

Minimize ||x|| among solutions to A x = b

Generically, if the columns is less than the number of rows then there are no solutions.

Usually there is more data than variables, so there are no solutions.

Always a least squares solution.

Solving least squares by hand

Use the formula:

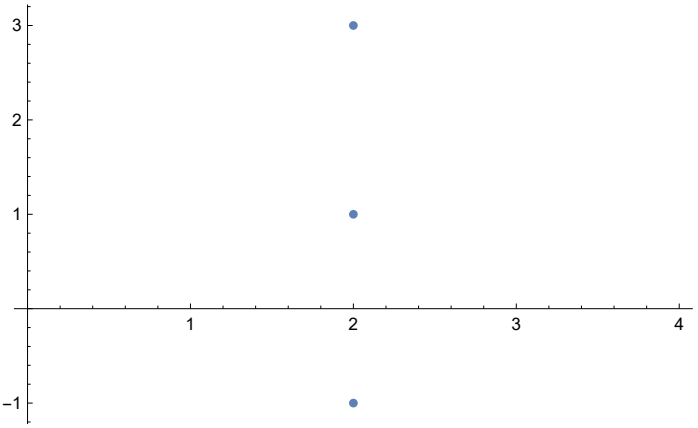
$$x = (A^t A)^{-1} A^t b$$

This formula only works when $A^t A$ is invertible. At least it is always a square matrix.

In most practical applications it is invertible.

In the case of linear regression, the matrix $A^t A$ is only singular when the points are degenerate, e.g. all x-values are the same.

Out[ ]=



Here we would solve

Out[ ]=

$$\begin{pmatrix} 2 & 1 \\ 2 & 1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} m \\ b \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \\ 3 \end{pmatrix}$$

but we can't do least squares with this formula because the matrix $A^t A$ is singular

Out[ ]=

$$\begin{pmatrix} 2 & 2 & 2 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 2 & 1 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 12 & 6 \\ 6 & 3 \end{pmatrix}$$

$$\det \begin{pmatrix} 12 & 6 \\ 6 & 3 \end{pmatrix} = 0$$

Instead least squares works using the SVD method.

Setting up Least Squares

Typically one wants to find the parameters that give a best fit.

y = m x + b

The data is x and y, and there could be a lot of data.

The parameters are m and b. There can only be one choice of parameters so least squares is a common choice.

Step 1. Identify the unknowns. What are you solving for? Usually these are the parameters of the problem.

The unknown parameters form the vector x in A x = b. In the end, this will be the computer output of least squares.

Out[•]= $A \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = b$

Step 2. Write down the equations if everything worked perfectly and there was no error.

y = α x + β for example, an use your actual data to write one or two of these.

5 = 3 α + β
4 = 2 α + β
....

The coefficients of the parameters from step 1 are going to be the rows of A.

Write these equations using the dot product.

Out[•]=
$$5 = \begin{pmatrix} 3 & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$
$$4 = \begin{pmatrix} 2 & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

Step 3. Convert the system of equations into a matrix equation by putting the columns together into matrices and vectors.

Out[•]= $\begin{pmatrix} 5 \\ 4 \end{pmatrix} = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$

Usually there is a lot of data so you want to automate the process of producing the matrix A and the vector b.

Step 4. One you have A and b, use the least squares function to find the vector of the parameters.

Singular Value Decomposition

While this is technically a decomposition of a matrix A it has many applications.

$A = U S V^t$

If A is m by n, then
U is a m by m orthogonal matrix.
S is a m by n rectangular diagonal matrix.
V is a n by n orthogonal matrix.

Singular Values

The singular values are decreasing, and the diagonal entries of S.
$s_{11} = s_1$ etc
$s_1 \geq s_2 \geq s_3 \geq \ldots$
s_1 is the maximal stretching done by A.

Orthogonal Matrices

$X X^t = I$

The rows form an orthonormal frame of vectors. So do the columns.

One projects onto an orthonormal frame by getting coordinates using the dot product.

Vector formulation

$A v_i = s_i u_i$

where u, v are rows of U and V
u_i are an orthonormal frame in the image space
v_i are an orthonormal frame in the domain space
$s_i \geq 0$
$s_1 \geq s_2 \geq s_3 \geq \ldots$

Other Factorizations besides SVD

Diagonalizing a Matrix

For a square matrix A.

$A = B D B^{-1}$

where D is diagonal.

Not always possible, but for the generic A it is possible.

If it is possible then the diagonal elements are the eigenvalues, and
$\det A = \Pi \lambda_i$

Defining a matrix via a transformation between bases

$A = B X B^{-1}$

Here X need not be diagonal.

A and X are said to be similar.

Quadratic Form for A

Q(v) = < A v, A v >

Consider 2 columns and 3 rows.

```
MatrixForm[A2]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} -3 & 2 \\ 2 & -2 \\ 3 & 1 \end{pmatrix}$$

```
A2.{x, y}
```

Out[•]= $\{-3\,x+2\,y,\ 2\,x-2\,y,\ 3\,x+y\}$

The quadratic form $< A2 \cdot v,\ A2 \cdot v >$

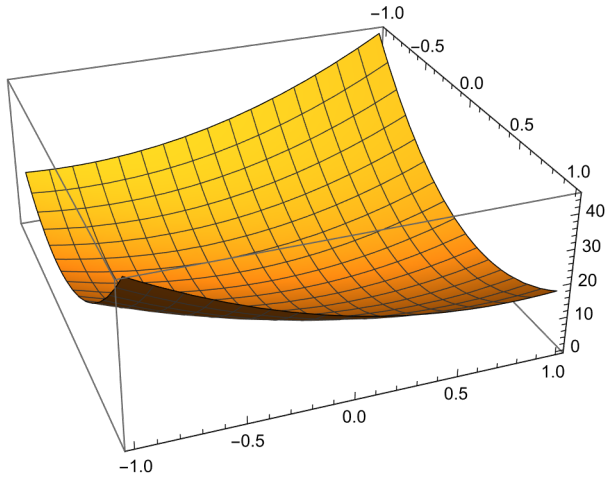
```
(A2.{x, y}).(A2.{x, y})
```

Out[•]= $(2\,x-2\,y)^2+(3\,x+y)^2+(-3\,x+2\,y)^2$

```
ExpandAll[(A2.{x, y}).(A2.{x, y})]
```

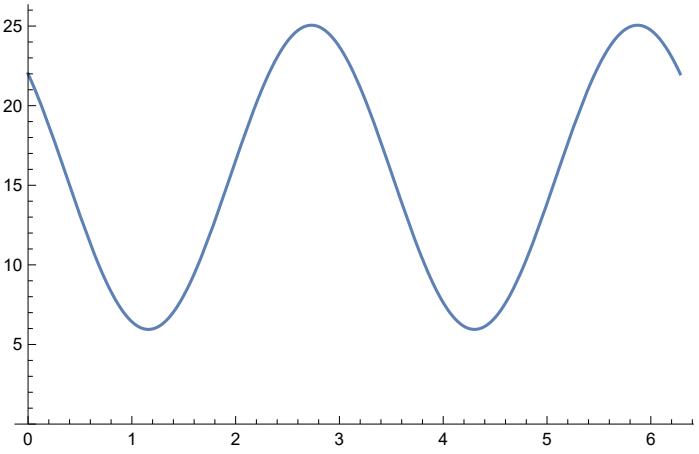
Out[•]= $22\,x^2-14\,x\,y+9\,y^2$

Out[•]=



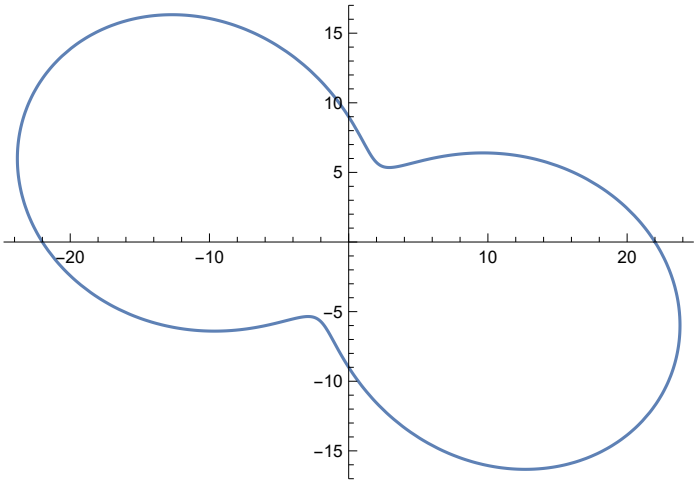
Plot it on the circle.

Out[ ]=



Polar Plot.

Out[ ]=

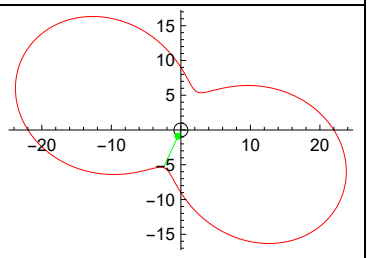
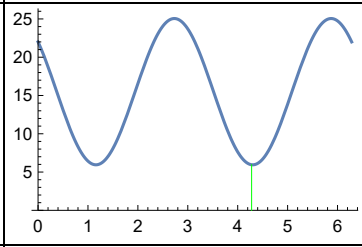
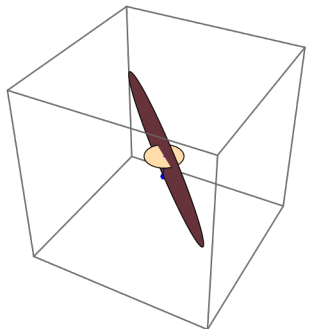


Because the domain is 2D and the image is in 3D we can visualize the inputs and outputs.

Here t0 is the angle on the unit circle.

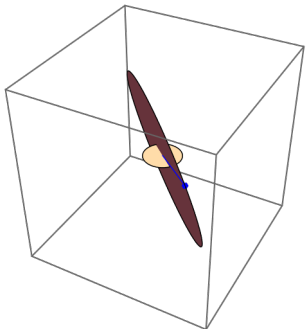
Out[ ]=

t0

angle	4.27885
polar plot	
quadratic form	
image	

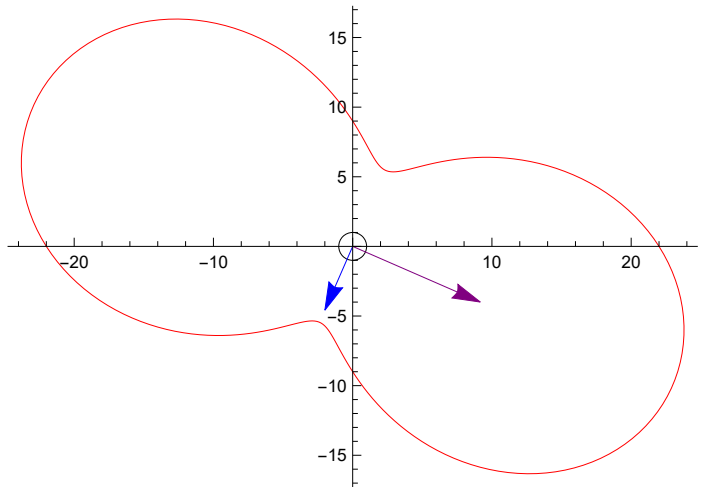
Out[ ]=

t0

angle	1.70903
image	

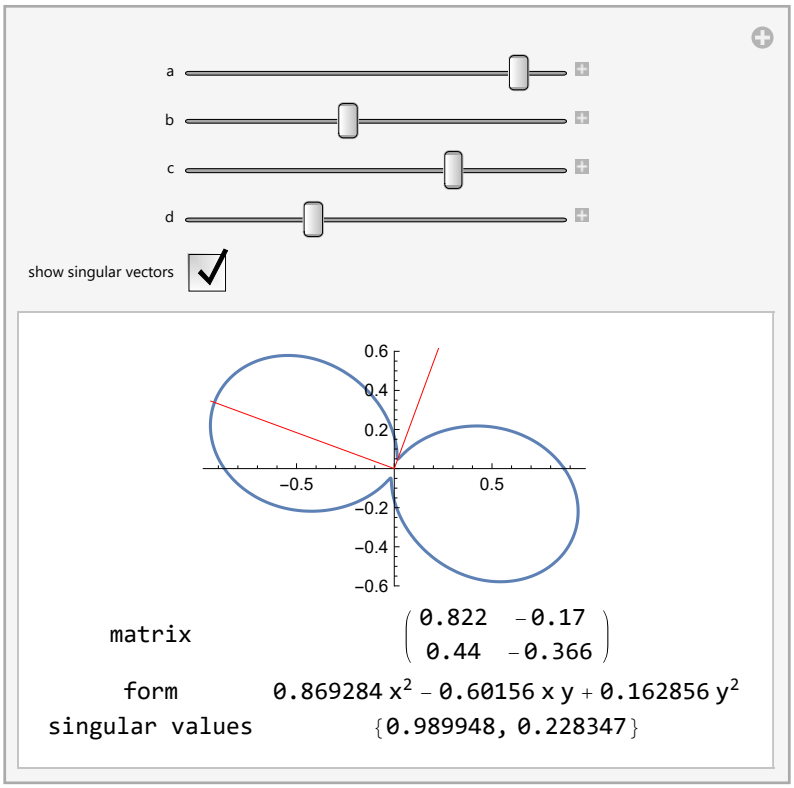
The singular vectors correspond to the minimum and the maximum along the unit circle.

Out[ ]=



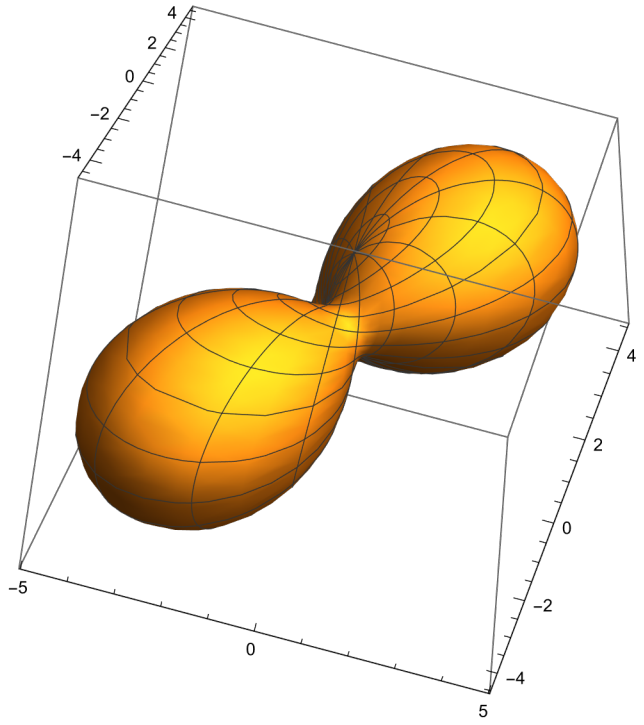
Consider possible 2x2 matrices:

Out[ ]=



If the domain is 3D then there is a third perpendicular vector.

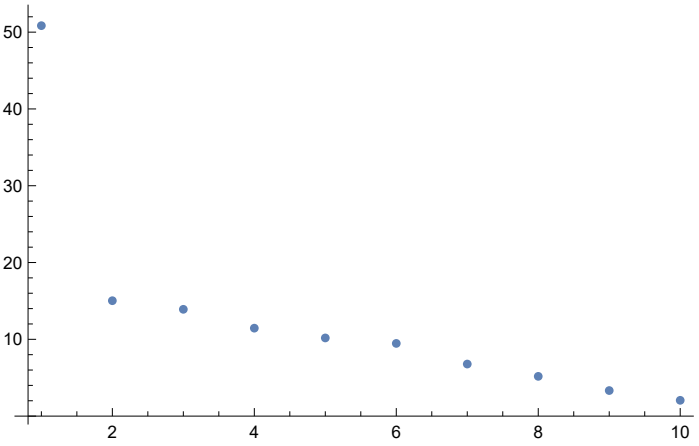
Out[ ]=



Singular values

Generically, they drop off quickly.

Out[ ]=



Max and Min ⟨ A · v, A · v ⟩

The singular vectors are where the rank of the Jacobian drops.

The singular values are the values of ||A v|| at those singular vectors.

Eigenvalues of $A \cdot A^t$ and $A^t \cdot A$

If A is m by n then A A^t is m by m, and a symmetric matrix.
Theory says that it has m eigenvalues.

Likewise $A^t A$ is n by n, symmetric, and has n eigenvalues.

Which ever is smaller, n or m, the eigenvalues of these two matrices are the same, the remainder of the eigenvalues are 0.

These eigenvalues are nonnegative, since they correspond to values of the quadratic form, so we can take their square root.

The square roots of these eigenvalues are the singular values of A.

Applications of Singular Value Decomposition

Practical Least Squares

First suppose that the target vector b is not in the image.

If one restricts to nonzero singular values, the vectors u_i are a basis for the image of A. E.g. suppose $s_3 > 0$ and $s_4 = 0$ then the nonzero singular values are s_1, s_2, s_3 and the image of A has an orthonormal basis u_1, u_2, u_3 Note the rank of A would be 3.

The orthogonal projection of a target vector b is the vector
$c_1 u_1 + c_2 u_2 + c_3 u_3$
in the image of A where
$c_i = < b,\ u_i >$ are coefficients coming from the dot product.

The SUV decomposition in vector form
$A v_i = s_i u_i$
gives the solution to the least squares
A x = b
via
$x = c_1/s_1 v_1 + c_2/s_2 v_2 + c_3/s_3 v_3$

On the other hand, suppose that b is in the image of A.

Follow the above procedure and then use the fact that the $v_i$are perpendicular to the null space to see that

$x = c_1/s_1 v_1 + c_2/s_2 v_2 + c_3/s_3 v_3$

minimizes ||x|| among solutions, because any two solutions differ by a null vector.

Image Processing

Compression

Recognition

Eigenfaces

Dimension Reduction

Clustering

Optimization

Principal Component Analysis

Data mining

Matrix Approximation

Weather Example

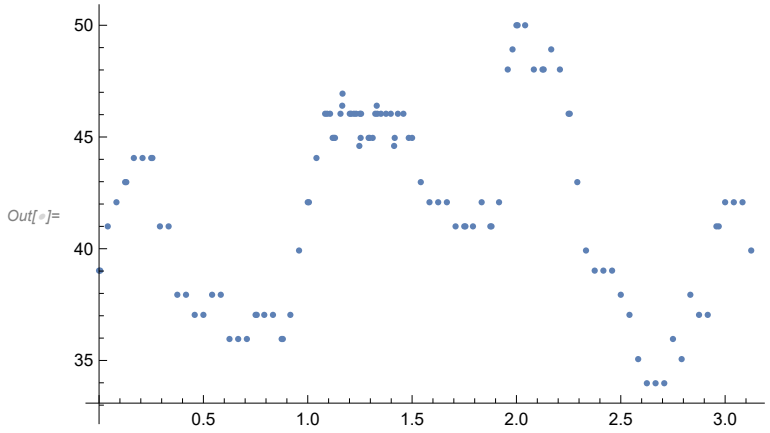
Temperature Data

Visualization

```
In[ ]:= Length[temperaturedata]
Out[ ]= 110
```

In[•]:=
```
ListPlot[temperaturedata]
```

Out[•]=

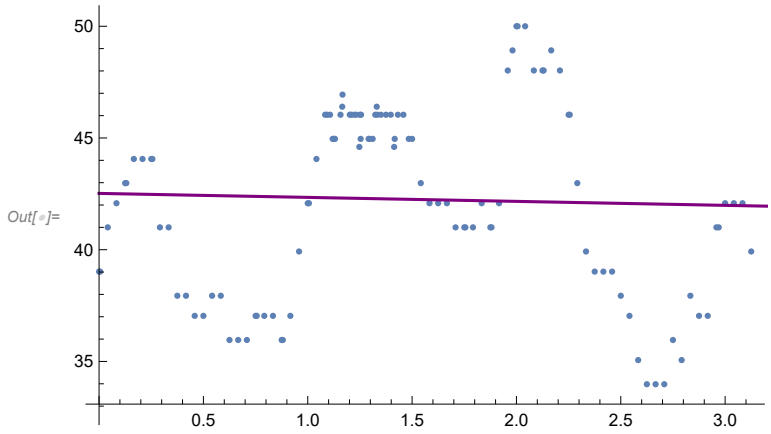


In[•]:=
```
Fit[temperaturedata, {1, x}, x]
```

Out[•]=
```
42.5199 - 0.178416 x
```

In[•]:=
```
Show[ListPlot[temperaturedata],
 Plot[42.51993373984794` - 0.1784157702383645` x, {x, 0, 4}, PlotStyle → Purple]]
```

Out[•]=

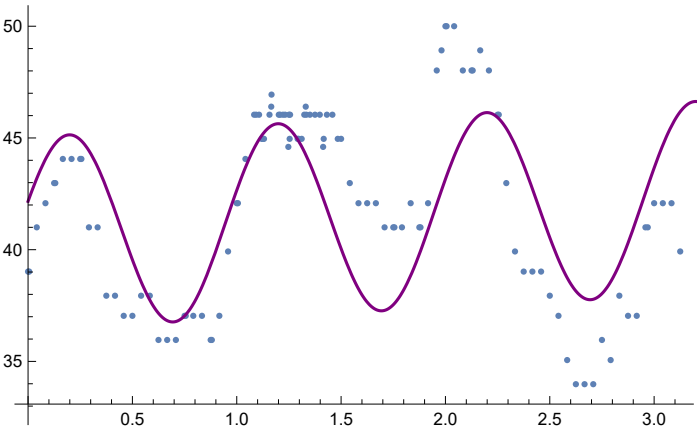


In[•]:=
```
Fit[temperaturedata, {1, x, Sin[2 Pi * x], Cos[2 Pi * x]}, x]
```

Out[•]=
```
40.7304 + 0.49765 x + 1.43172 Cos[2 π x] + 4.0653 Sin[2 π x]
```

In[•]:=
```
Show[ListPlot[temperaturedata],
 Plot[40.73039624498475` + 0.49764999292087386` x + 1.4317217383214533` Cos[2 π x] +
   4.065299401120508` Sin[2 π x], {x, 0, 4}, PlotStyle → Purple]]
```

Out[•]=



Setting up least squares

Find the Parameters

$y = \alpha_1 x + \alpha_2 \sin x + \alpha_3 \cos x + \alpha_4$

A linear combination of 1, sin x, cos x, x

The parameters are the coefficients.

$[\alpha_1, \alpha_2, \alpha_3, \alpha_4]$

Write the Equation as a dot product

$y = [\alpha_1, \alpha_2, \alpha_3, \alpha_4] \cdot [x, \sin x, \cos x, 1]$

The second vector is going to be a row of A

The y value is the corresponding element of the vector b

Write as a matrix equation

Usually, but not always, the rows of A are going to be a function of the dependent variable.

In[•]:=
```
ARow[{x_, y_}] := {x, Sin[x], Cos[x], 1}
```

In[•]:=
```
BRow[{x_, y_}] := y
```

In[•]:=
```
A = Map[ARow, temperaturedata];
```

In[•]:=
```
b = Map[BRow, temperaturedata];
```

In[•]:=
```
MatrixForm[A]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 0. & 0. & 1. & 1 \\ 0.00555556 & 0.00555553 & 0.999985 & 1 \\ 0.0416667 & 0.0416546 & 0.999132 & 1 \end{pmatrix}$$

0.0416667	0.0416546	0.999132	1
0.0833333	0.0832369	0.99653	1
0.125	0.124675	0.992198	1
0.130556	0.130185	0.99149	1
0.166667	0.165896	0.986143	1
0.208333	0.20683	0.978377	1
0.25	0.247404	0.968912	1
0.255556	0.252783	0.967523	1
0.291667	0.287549	0.957766	1
0.333333	0.327195	0.944957	1
0.375	0.366273	0.930508	1
0.416667	0.404715	0.914443	1
0.458333	0.442454	0.896791	1
0.5	0.479426	0.877583	1
0.541667	0.515565	0.856851	1
0.583333	0.550809	0.834631	1
0.625	0.585097	0.810963	1
0.666667	0.61837	0.785887	1
0.708333	0.650569	0.759447	1
0.75	0.681639	0.731689	1
0.755556	0.685693	0.727891	1
0.791667	0.711525	0.70266	1
0.833333	0.740177	0.672412	1
0.875	0.767544	0.640997	1
0.880556	0.771093	0.636723	1
0.916667	0.793578	0.608469	1
0.958333	0.818235	0.574885	1
1.	0.841471	0.540302	1
1.00556	0.84446	0.535619	1
1.04167	0.863247	0.504782	1
1.08333	0.883524	0.468386	1
1.09306	0.888036	0.459774	1
1.10694	0.894336	0.447396	1
1.11944	0.899858	0.436182	1
1.125	0.902268	0.431177	1
1.13056	0.904649	0.426157	1
1.15694	0.915579	0.402139	1
1.16528	0.918898	0.394495	1
1.16667	0.919445	0.393219	1
1.20139	0.932541	0.361063	1
1.20833	0.935026	0.354578	1
1.22153	0.939623	0.342211	1
1.23194	0.943137	0.332404	1
1.24722	0.948105	0.317957	1
1.25	0.948985	0.315322	1
1.25347	0.950074	0.312025	1
1.25556	0.950722	0.310045	1
1.29167	0.961296	0.275519	1
1.29653	0.962624	0.270843	1
1.31111	0.966471	0.256776	1
1.32361	0.969605	0.244676	1

1.33056	0.971281	0.237936	1
1.33333	0.971938	0.235238	1
1.35069	0.975875	0.218329	1
1.375	0.980893	0.194548	1
1.39792	0.985093	0.17202	1
1.41389	0.987715	0.156264	1
1.41667	0.988146	0.15352	1
1.43194	0.990376	0.138406	1
1.45833	0.993683	0.112226	1
1.48403	0.996238	0.0866597	1
1.5	0.997495	0.0707372	1
1.54167	0.999576	0.0291255	1
1.58333	0.999921	−0.0125367	1
1.625	0.998531	−0.0541771	1
1.66667	0.995408	−0.0957235	1
1.70833	0.990557	−0.137104	1
1.75	0.983986	−0.178246	1
1.75556	0.982981	−0.18371	1
1.79167	0.975707	−0.219079	1
1.83333	0.965735	−0.259531	1
1.875	0.954086	−0.299534	1
1.88056	0.952407	−0.304829	1
1.91667	0.940781	−0.339016	1
1.95833	0.925843	−0.377909	1
1.98125	0.91694	−0.399025	1
2.	0.909297	−0.416147	1
2.00556	0.906971	−0.421192	1
2.04167	0.891174	−0.453662	1
2.08333	0.871503	−0.49039	1
2.125	0.85032	−0.526266	1
2.13056	0.847383	−0.530982	1
2.16667	0.82766	−0.561229	1
2.20833	0.803564	−0.595218	1
2.25	0.778073	−0.628174	1
2.25556	0.774571	−0.632487	1
2.29167	0.751232	−0.660039	1
2.33333	0.723086	−0.690758	1
2.375	0.693685	−0.720278	1
2.41667	0.66308	−0.748549	1
2.45833	0.631324	−0.775519	1
2.5	0.598472	−0.801144	1
2.54167	0.564581	−0.825377	1
2.58333	0.529711	−0.848178	1
2.625	0.49392	−0.869507	1
2.66667	0.457273	−0.889327	1
2.70833	0.419831	−0.907602	1
2.75	0.381661	−0.924302	1
2.79167	0.342828	−0.939398	1
2.83333	0.3034	−0.952863	1
2.875	0.263446	−0.964674	1
2.91667	0.223034	−0.974811	1

$$\begin{pmatrix} 2.95833 & 0.182235 & -0.983255 & 1 \\ 2.96944 & 0.171299 & -0.985219 & 1 \\ 3. & 0.14112 & -0.989992 & 1 \\ 3.04167 & 0.0997598 & -0.995012 & 1 \\ 3.08333 & 0.0582264 & -0.998303 & 1 \\ 3.125 & 0.0165919 & -0.999862 & 1 \end{pmatrix}$$

In[•]:= **MatrixForm[b]**

Out[•]//MatrixForm=

$$\begin{pmatrix} 39.02 \\ 39.02 \\ 41. \\ 42.08 \\ 42.98 \\ 42.98 \\ 44.06 \\ 44.06 \\ 44.06 \\ 44.06 \\ 41. \\ 41. \\ 37.94 \\ 37.94 \\ 37.04 \\ 37.04 \\ 37.94 \\ 37.94 \\ 35.96 \\ 35.96 \\ 35.96 \\ 37.04 \\ 37.04 \\ 37.04 \\ 37.04 \\ 35.96 \\ 35.96 \\ 37.04 \\ 39.92 \\ 42.08 \\ 42.08 \\ 44.06 \\ 46.04 \\ 46.04 \\ 46.04 \\ 44.96 \\ 44.96 \\ 44.96 \\ 46.04 \\ 46.4 \\ 46.94 \\ 46.04 \end{pmatrix}$$

```
46.04
46.04
46.04
 44.6
46.04
44.96
46.04
44.96
44.96
44.96
46.04
 46.4
46.04
46.04
46.04
46.04
 44.6
44.96
46.04
46.04
44.96
44.96
42.98
42.08
42.08
42.08
 41.
 41.
 41.
 41.
42.08
 41.
 41.
42.08
48.02
48.92
 50.
 50.
 50.
48.02
48.02
48.02
48.92
48.02
46.04
46.04
42.98
39.92
39.02
39.02
39.02
```

```
37.94
37.04
35.06
33.98
33.98
33.98
35.96
35.06
37.94
37.04
37.04
 41.
 41.
42.08
42.08
42.08
39.92
```

Run Least Squares

The output is the least squares solution x to A x = b

```
In[•]:= LeastSquares[A, b]
Out[•]= {-6.91985, 6.93377, -8.61478, 48.3644}
```

These are the parameters $[\alpha_1, \alpha_2, \alpha_3, \alpha_4]$

Note that the convention of the order is set in step 1.

Comparing Eigenvalues and Singular Values

Eigenvalues ≠ Singular Values

Only if it is a square matrix and it is symmetric.

Singular values from Eigenvalues for $A\,A^t$

It is not hard to show that for $A\,A^t$ the eigenvalues λ are real and satisfy $\lambda \geq 0$

Then $s_i = \sqrt{\lambda_i}$ are the singular values.

Note that A(t) = t A makes λ scale like t^2

Eigenvalues of $A^t\,A$

If $\lambda \neq 0$ is an eigenvalue of $A^t\,A$ then it is also an eigenvalue of $A\,A^t$
If v is the corresponding eigenvector for $A^t\,A$ then A v is the corresponding eigenvector of $A\,A^t$

Same for switching the roles of A and A^t

Extra eigenvalues are 0

If A is m by n then
$A\,A^t$ is m by m
$A^t\,A$ is n by n

If n≠m then the extra |n-m| eigenvalues of these square matrices are 0.
If n≠m then the extra |n-m| singular values of A are 0.

When a matrix multiplication is iterated, then eigenvalues

A v = λ v

It's the same vector v

When size matters, then singular values

Low rank approximation

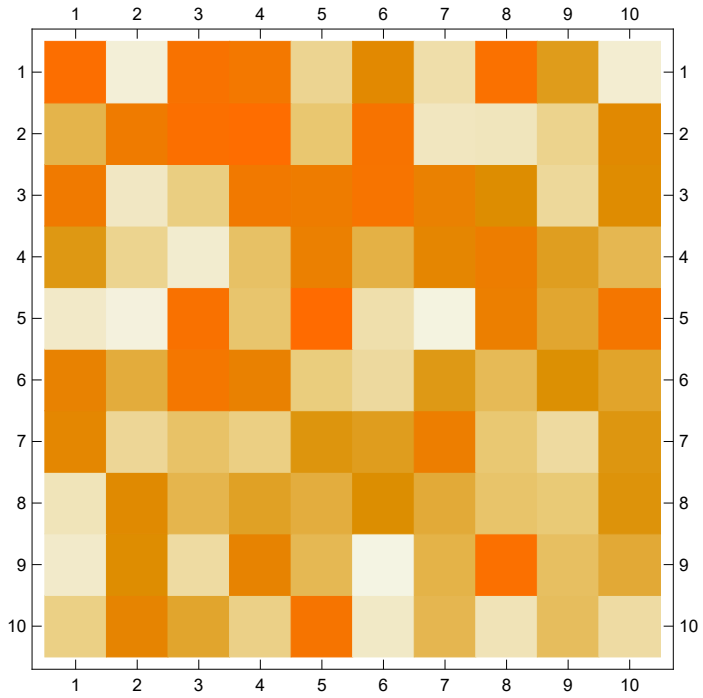
Take the first k columns of V and the first k of U.

Take an example:

In[15]:=
```
A = RandomReal[1, {10, 10}];
```

In[18]:=
```
MatrixPlot[A]
```

Out[18]=



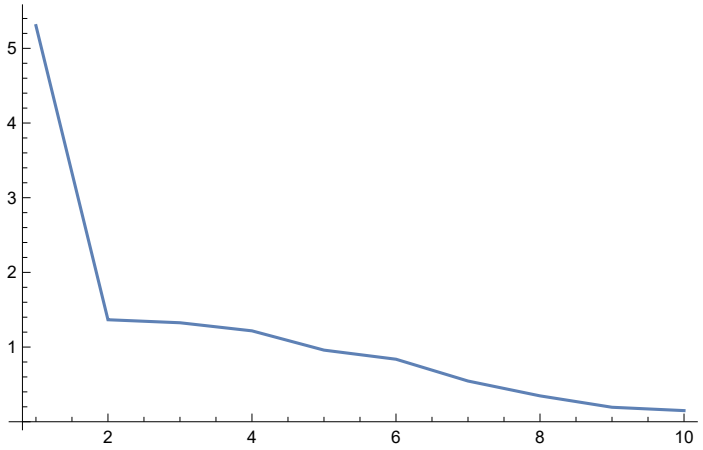
In[16]:=
```
SUV = SingularValueDecomposition[A];
```

In[17]:=
```
{U, S, V} = SUV;
```

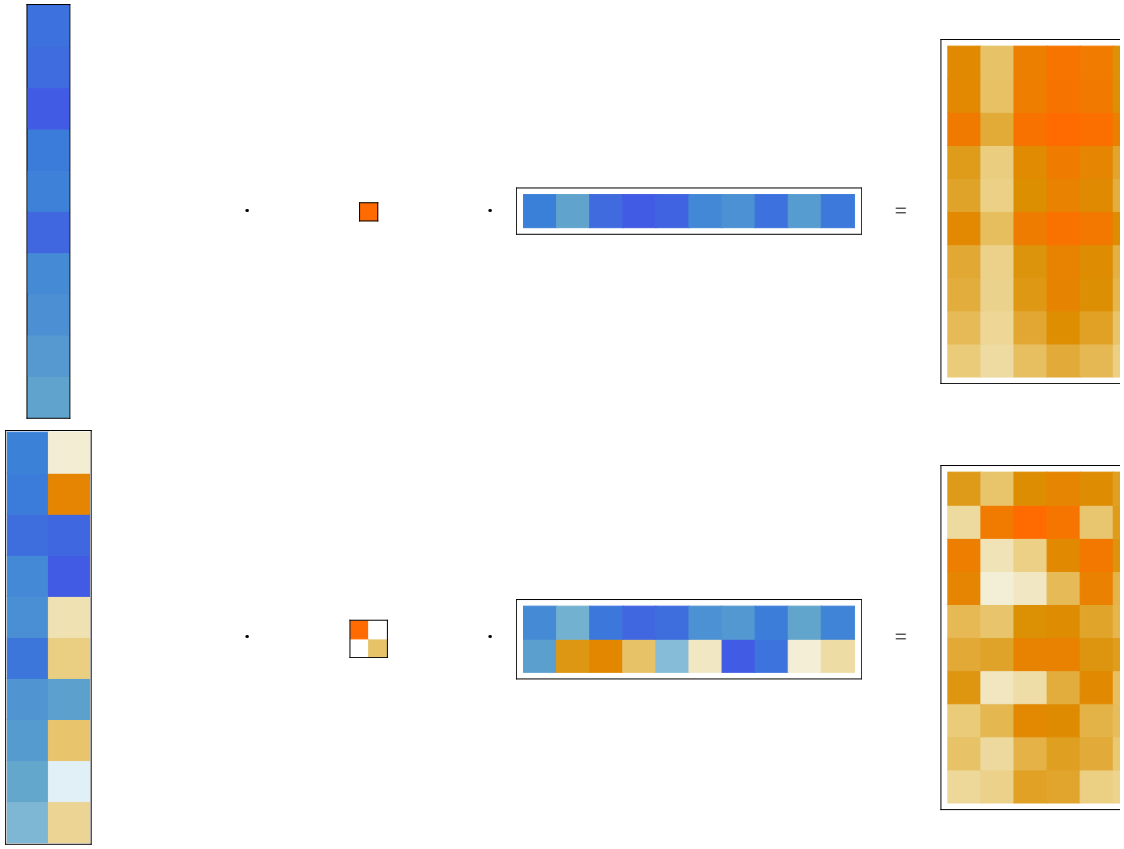
In[25]:=
```
A0[i_] := U[[All, ;; i]].S[[ ;; i, ;; i]].Transpose[V[[All, ;; i]]]
```

In[26]:=
```
ListLinePlot[Diagonal[S], PlotRange → All]
```

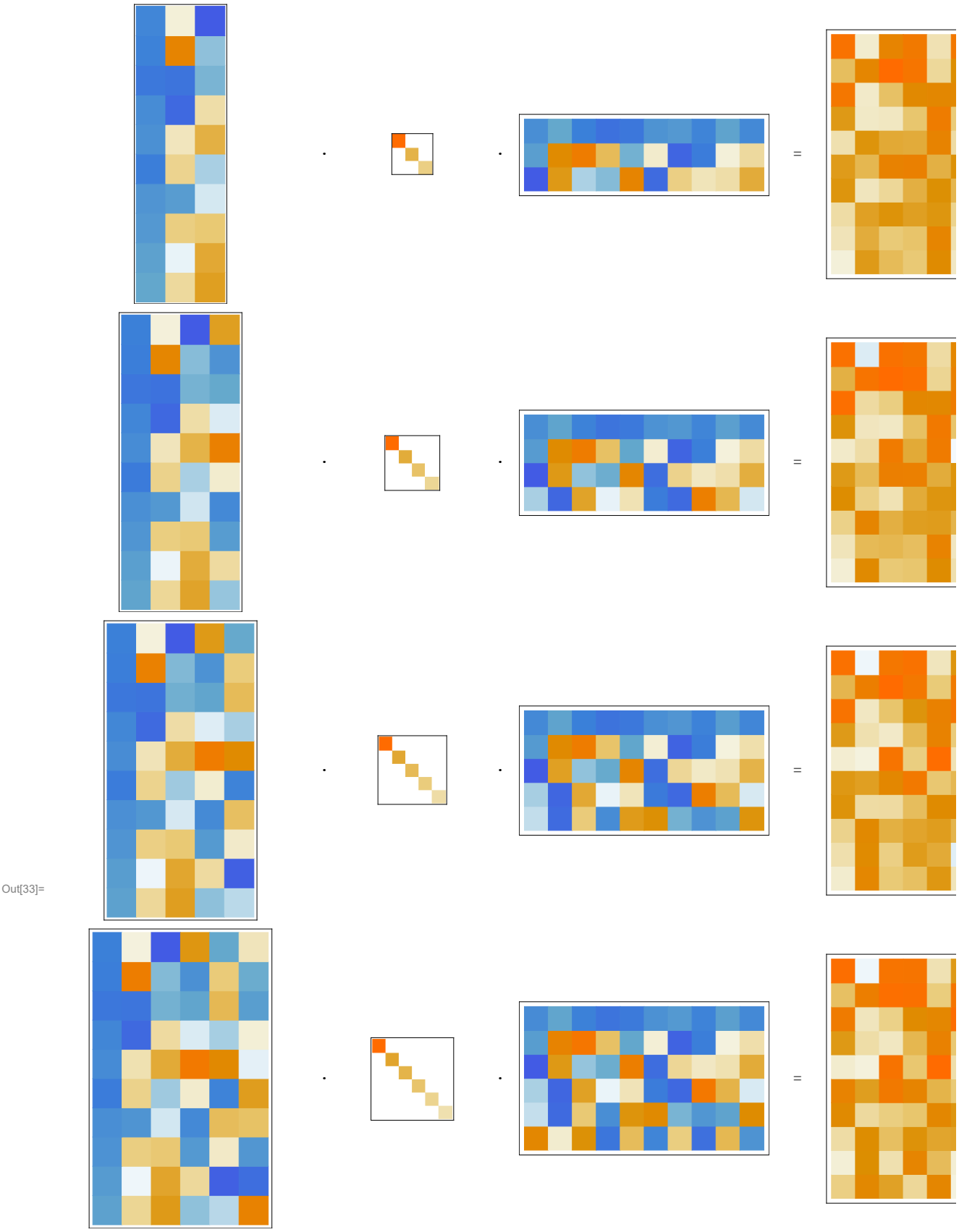
Out[26]=

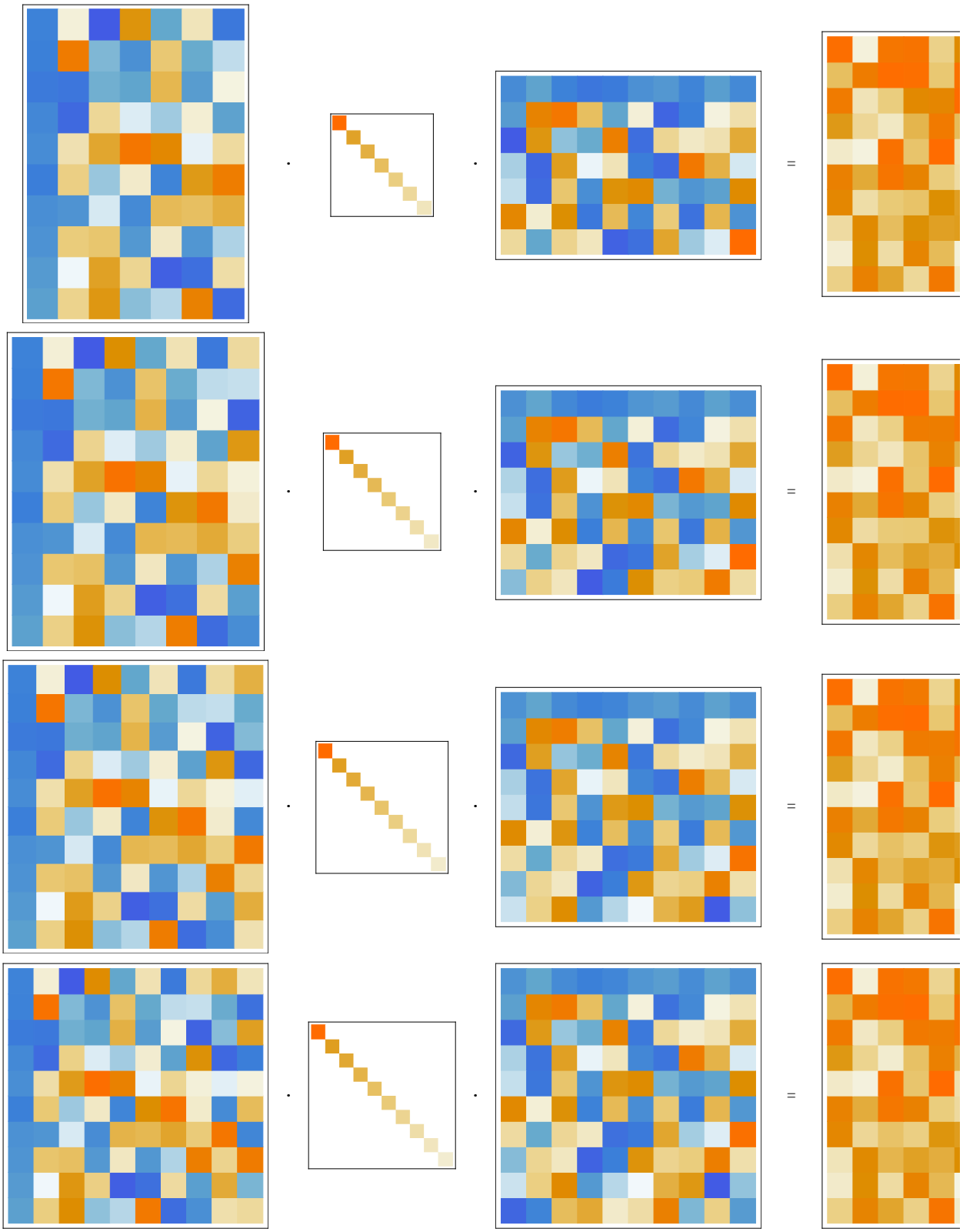


In[33]:=
```
Grid[Table[{MatrixPlot[U[[All, ;; i]], FrameTicks → None],
    "·", MatrixPlot[S[[ ;; i, ;; i]], ImageSize → 10 i, FrameTicks → None],
    "·", MatrixPlot[Transpose[V[[All, ;; i]]], FrameTicks → None],
    " = ", MatrixPlot[A0[i], FrameTicks → None]}, {i, 10}]]
```



Out[33]=





Low rank factorization

X = A B

For example, to compress the data in X, we write it as the product of two low rank matrices.

If X is m by n it has m n entries

If A is m by k and B is k by n then there are
m k + k n

So if k is much smaller than m and n, there are a lot less to keep track of.

Take m = n = 300, e.g., a grayscale image with 300 rows and columns.
Take k = 10.

In[10]:=
```
300 * 300
```

Out[10]=
```
90 000
```

In[13]:=
```
300 * 10 + 10 * 300
```

Out[13]=
```
6000
```

Principal Components

As this is a statistical method, one subtracts off the average values first.

Dimension Reduction

Take m points in n-dimensions.
Reduce the dimension to 2.
Use a 2 by n matrix.

The matrix comes from the first two rows of V, so it is an orthogonal projection.

In[38]:=
```
labels = ExampleData["AerialImage"][[All, 2]]
```

Out[38]=
```
{Earth, FosterCity, Oakland, Oakland2, Pentagon, Richmond, Richmond2, SanDiego,
 SanDiego2, SanDiego3, SanDiego4, SanDiego5, SanDiego6, SanDiego7, SanDiego8,
 SanDiego9, SanDiego10, SanDiego11, SanDiego12, SanDiego13, SanFrancisco,
 SanFrancisco2, SanFrancisco3, SanFrancisco4, SanFrancisco5, SanFrancisco6,
 SanFrancisco7, SanFrancisco8, SanFrancisco9, Shreveport, Stockton, Stockton2,
 Stockton3, Stockton4, Stockton5, Stockton6, WashingtonIR, WoodlandHills}
```

In[40]:=
```
Map[ImageDimensions, ExampleData /@ ExampleData["AerialImage"]]
```

Out[40]=
```
{{512, 512}, {512, 512}, {512, 512}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024},
 {512, 512}, {512, 512}, {512, 512}, {512, 512}, {512, 512}, {512, 512}, {512, 512},
 {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024},
 {512, 512}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024},
 {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024},
 {1024, 1024}, {1024, 1024}, {1024, 1024}, {1024, 1024}, {2250, 2250}, {512, 512}}
```

In[45]:=
```
data = Take[Flatten[#], 262 000] & /@
   Map[ImageData, ImageResize[ExampleData[#], {512, 512}] & /@ ExampleData["AerialImage"]];
```

In[46]:=
```
Dimensions[data]
```

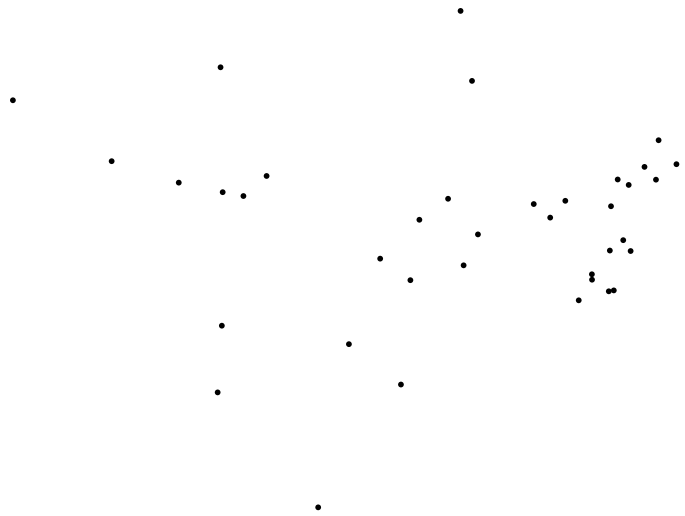
Out[46]=
```
{38, 262 000}
```

In[47]:=
```
points = DimensionReduce[data, 2]
```

Out[47]=
```
{{-837.374, 238.817}, {318.819, -26.2271}, {-200.827, -223.213},
 {385.817, 163.105}, {-444.016, 301.278}, {259.441, -90.8201}, {293.466, -45.8872},
 {-523.162, 82.7558}, {32.3587, 275.511}, {-400.683, 57.4065}, {359.113, 112.6},
 {234.517, -140.165}, {380.845, 88.3789}, {329.173, 78.4127}, {-650.305, 123.398},
 {300.958, -121.383}, {419.647, 117.688}, {-439.969, 64.6734}, {-356.795, 95.3888},
 {332.84, -46.7787}, {-259.051, -532.368}, {295.665, 38.0333}, {291.481, -123.094},
 {-141.606, -61.2352}, {16.4574, -73.8723}, {-84.3837, -102.03},
 {-12.9957, 52.2231}, {308.353, 88.7447}, {259.716, -101.121}, {-67.3457, 12.4255},
 {180.471, 16.5172}, {-441.561, -188.228}, {149.316, 42.2046}, {209.115, 48.3575},
 {43.5988, -15.2899}, {-449.495, -314.583}, {-102.271, -299.706}, {10.6734, 408.083}}
```

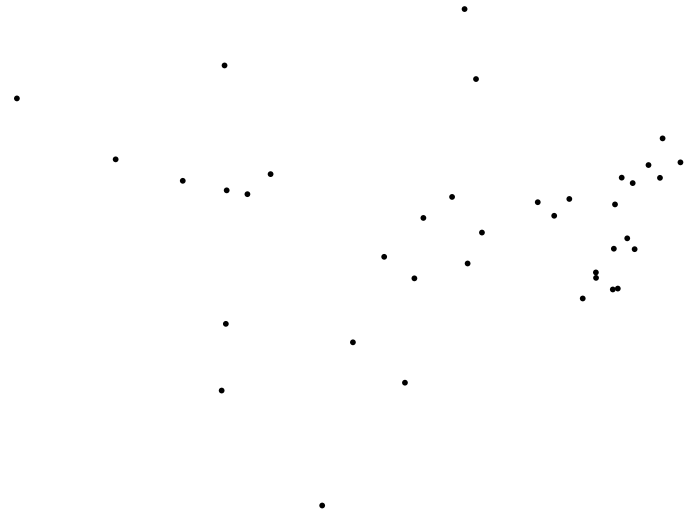
In[48]:=
```
Graphics[Point[points]]
```

Out[48]=



In[49]:=
```
Graphics[MapThread[Tooltip[Point[#], #2] &, {points, labels}]]
```

Out[49]=



In[50]:=
```
Graphics[MapThread[Text[#2, #] &, {points, labels}]]
```

Out[50]=